

Analizador léxico y sintáctico de C-TDS

Diseño e interfaz del AST

Agustin Alieni, Fran Natale y Julian Varea

1. Introducción

En esta etapa construimos el analizador léxico (`lexer.l`, con Flex) y el analizador sintáctico (`bison.y`, con Bison) de C-TDS, y dejamos definida la interfaz del árbol de sintaxis abstracta (AST) que va a sostener las etapas siguientes del compilador (análisis semántico, generación de código intermedio y, finalmente, `assembly`). En esta entrega no implementamos el AST: dejamos declarada su interfaz en `ast.h`, con los cuerpos de `ast.c` vacíos, y las acciones semánticas de la gramática comentadas, a la espera de que terminemos de cerrar en equipo algunas decisiones de diseño que todavía están abiertas.

2. Analizador léxico

El lexer reconoce todos los tokens que describe la especificación del lenguaje: las palabras reservadas (`boolean`, `int`, `float`, `void`, `if`, `else`, `while`, `return`), los identificadores, las constantes numéricas (enteras y de punto flotante), las constantes booleanas, los operadores aritméticos, relacionales y lógicos, los delimitadores, y los comentarios de línea y multilinea.

2.1. `main` no es palabra reservada

Decidimos no tratar `main` como palabra reservada, a pesar de que el lenguaje exige que todo programa tenga un método con ese nombre. La razón es que esa exigencia es una regla semántica (debe existir un método llamado `main` con una firma particular), no una regla léxica ni sintáctica: en la gramática, `main` es simplemente el identificador de un método como cualquier otro. Si lo hubiéramos declarado como palabra reservada, un identificador de esa forma dejaría de poder usarse como nombre de variable o de cualquier otro método, lo cual no está pedido en ningún lado de la especificación. Por eso `main` matchea la regla general de `ID`, y la verificación de que exista un método `main` con la firma correcta queda para el análisis semántico.

2.2. Número de línea en los mensajes de error

Activamos `%option yylineno` y lo usamos tanto en los mensajes de error léxico del propio `lexer.l` como en `yyerror` del lado de Bison. Nos pareció imprescindible para que un mensaje de error sea útil: sin número de línea, encontrar dónde está el problema en un archivo fuente de varias decenas de líneas obliga a revisar todo el archivo a mano.

2.3. Comentarios multilinea

Reconocemos comentarios multilinea con la expresión regular `/*([^\*]|\\*+[^*/])*\*+/*`, que exige que el comentario esté efectivamente cerrado con `*/`. El caso de un comentario multilinea que nunca se cierra (el archivo termina dentro de un comentario abierto) queda señalado como algo pendiente de resolver en una iteración posterior del lexer: hoy ese caso no genera un mensaje de error específico, y preferimos dejarlo así por el momento en lugar de introducir un estado exclusivo (`%x`) y una regla de «EOF» sin haberlo discutido primero en equipo.

3. Analizador sintáctico

3.1. Un conflicto shift/reduce que nos obligó a rediseñar la gramática

La primera versión de la gramática que escribimos separaba las declaraciones globales en dos listas independientes, una para variables y otra para métodos:

```
Program : VariableDeclarations MethodDeclarations ;
```

Esta estructura calca directamente el orden en que aparecen en la letra los no terminales `<var_decl>` y `<method_decl>`, pero no funciona en la práctica: al probarla contra el programa de ejemplo de la propia especificación del lenguaje, que declara una variable global y a continuación un método con retorno no-void,

```
int a;
int sumar(int a, int b) { }
```

el parser fallaba con un error de sintaxis apenas encontraba el segundo `int`. La causa es un conflicto de desplazamiento/reducción (shift/reduce) inherente a un parser LALR(1), que solo tiene un token de adelanto (lookahead) para decidir. Tanto una declaración de variable como una de método empiezan igual, con `Type ID`, y Bison, ante la duda, siempre prefiere desplazar antes que reducir: al ver el segundo `int` todavía dentro de `VariableDeclarations`, asumía que se trataba de otra variable y esperaba un `;` o una `,`. Como en cambio encontraba un `(`, la lista de variables nunca llegaba a cerrarse y el parser no tenía forma de retroceder para reinterpretar esa declaración como un método. En la práctica, esto significa que ningún método con tipo de retorno no-void lograba parsearse si estaba precedido por al menos una variable global, lo cual invalidaba directamente el ejemplo de la letra.

La solución que aplicamos fue unificar ambas listas en una sola, dejando que la decisión de si `Type ID` arranca una variable o un método se tome recién al ver el token siguiente:

```
Declarations
: /* vacio */
| Declaration Declarations
;
```

```
Declaration
: VariableDeclaration
| MethodDeclaration
;
```

Con esta forma, el parser consume `Type ID` de manera neutra, sin comprometerse todavía a ninguna de las dos alternativas, y recién decide al ver el token siguiente: si es `;` o `,`, se trata de una variable; si es `(`, de un método. Esto permite además que variables y métodos aparezcan intercalados en el scope global, que es exactamente lo que hace el ejemplo de la letra. De paso, aprovechamos para sacar el no terminal intermedio `ReturnType` e integrar directamente `Type` y `VOID` en `MethodDeclaration`, para no introducir una reducción adicional que pudiera volver a confundir al parser en ese mismo punto.

Verificamos la corrección de este cambio de dos formas. Por un lado, generamos la gramática con `bison -d -v` y confirmamos que el archivo de reporte no muestra ningún conflicto shift/reduce ni reglas inalcanzables, algo que sí aparecía con el diseño original. Por otro lado, compilamos el parser completo contra Flex y contra la interfaz de AST y lo corrimos sobre el ejemplo de la letra y sobre variantes propias (variables intercaladas con métodos, condicionales anidados, ciclos, llamadas a método con y sin argumentos): todos los casos válidos parsean sin errores formales.

3.2. Precedencia de operadores

Para resolver la precedencia y asociatividad de los operadores de **Expression** usamos las declaraciones **%left** y **%right** de Bison, siguiendo la tabla de precedencia de menor a mayor que se desprende de la especificación del lenguaje: disyunción lógica, conjunción lógica, igualdad, relacionales, suma y resta, multiplicación/división/resto, y por último la negación lógica y el menos unario. Optamos por esta vía declarativa en lugar de reescribir **Expression** como una jerarquía explícita de no terminales por nivel de precedencia (una alternativa habitual cuando no se cuenta con declaraciones de precedencia de la herramienta). Es un punto que podríamos reconsiderar más adelante si en alguna etapa posterior necesitamos que la propia estructura del árbol refleje la precedencia de forma explícita, pero por ahora nos alcanza con que Bison la resuelva a partir de estas declaraciones.

4. Diseño del AST

4.1. Por qué separamos el AST de **bison.y**

Decidimos no definir la estructura del AST directamente en **bison.y**, sino en un archivo aparte, **ast.h**. La razón es que sobre este árbol todavía se va a construir el análisis semántico, un intérprete o generador de código intermedio, y finalmente el generador de assembly, y no queríamos que esas etapas dependieran de un archivo pensado en primer lugar para la gramática. Separar la interfaz del árbol en su propio header nos permite mantener **bison.y** enfocado en las reglas de producción y dejar la definición de los nodos como un contrato aparte, que cualquier etapa posterior puede incluir sin arrastrar nada de Bison.

4.2. Por qué **ast.h** solo tiene interfaz, y **ast.c** está vacío

Dentro de esa separación, distinguimos además interfaz de implementación: **ast.h** declara los tipos y las firmas de las funciones constructoras del árbol, documentando en comentarios qué hace cada una, quién es dueño de la memoria que devuelve y qué se espera de sus parámetros; **ast.c** contiene esas mismas funciones pero con el cuerpo vacío (un **TODO**). No las implementamos todavía porque hacerlo implica tomar decisiones de diseño que como equipo no terminamos de cerrar (por ejemplo, si el árbol va a liberar memoria a medida que se descarta, o cómo van a convivir exactamente los punteros **left/mid/right** con el arreglo **children[]** en los distintos tipos de nodo). Preferimos dejar esas firmas fijas y bien documentadas ahora, y resolver el cuerpo de cada función más adelante, cuando todo el equipo esté de acuerdo en cómo hacerlo, en lugar de que esa decisión quede tomada de forma implícita por quien haya escrito el primer código. Por eso en esta entrega **ast.h** es, en los hechos, toda la documentación de diseño del AST que tenemos: cada función está comentada con su propósito, sus parámetros y quién es responsable de liberar la memoria que devuelve, aunque todavía no haga nada.

Por el mismo motivo dejamos las acciones semánticas de **bison.y** escritas pero comentadas: cada regla de la gramática indica, a modo de comentario, cómo se vería la construcción del nodo correspondiente una vez que el AST esté implementado (qué **NodeType** le corresponde, qué va en **left/mid/right** o en **children[]**), pero no se ejecuta ningún código todavía. Esto nos permitió dejar registrada, regla por regla, la intención de diseño de cada construcción del lenguaje sin depender de que **ast.c** ya esté terminado, y sin arriesgarnos a que el parser genere nodos a medio definir o haga referencias a memoria sin inicializar. Cuando decidamos avanzar con la implementación del AST, el trabajo va a consistir en completar los cuerpos de **ast.c** siguiendo lo que ya documentamos en **ast.h**, y recién ahí descomentar estas acciones.

4.3. Estructura de NodeAST

Cada nodo del árbol (**NodeAST**) tiene un **NodeType** que indica qué construcción del lenguaje representa, un **Symbol** opcional para guardar el identificador o el valor asociado, punteros **left**, **mid** y **right** para los casos con una cantidad fija y chica de hijos (por ejemplo, una asignación o un **if-else**), y un arreglo **children[]** de tamaño variable para los casos donde la cantidad de hijos depende del programa fuente (por ejemplo, la lista de identificadores de una declaración de variables, o los argumentos de una llamada a método). El campo **type** guarda el tipo semántico del nodo (**TYPE_INT**, **TYPE_BOOL**, etc.), separado de **nodeType** para no mezclar "qué construcción sintáctica es este nodo" con "qué tipo de dato tiene", que son dos preguntas distintas y que además se resuelven en momentos distintos del proceso (la primera durante el parsing, la segunda recién en el análisis semántico).

4.4. Criterio para crear un nodo

No decidimos crear un nodo por cada no terminal de la gramática, sino únicamente para las construcciones que aportan información propia o que cambian el comportamiento del árbol resultante. Hay no terminales que existen en la gramática solo para organizar la sintaxis (evitar ambigüedades, factorizar alternativas) pero que en el AST no necesitan representación propia: por ejemplo, **Statement** agrupa todas las variantes de sentencia posibles, pero el nodo real lo termina aportando la alternativa concreta que **matcheó** (una asignación, un **return**, un bloque), así que **Statement** simplemente redirige el valor de su hijo hacia \$\$ sin crear un nodo nuevo. En cambio, **Block** sí tiene su propio **BLOCK_NODE**, porque no es un simple contenedor sintáctico: representa la apertura de un nuevo scope y agrupa las declaraciones de variables y las sentencias que contiene, que sí son información real que el árbol necesita conservar.

5. Alcance de esta entrega

Lo que dejamos funcionando en esta etapa es el análisis léxico y sintáctico completo de C-TDS: el parser reconoce todo el lenguaje descrito en la especificación (declaraciones de variables y métodos intercaladas, bloques, condicionales, ciclos, llamadas a método, expresiones con toda la precedencia de operadores) sin conflictos ni reglas inalcanzables, y compila junto con el analizador léxico sin errores. Del AST, en esta etapa solo dejamos definida la interfaz (**ast.h**) y las invocaciones ya escritas pero comentadas dentro de **bison.y**, a modo de referencia de cómo se va a construir cada nodo. No es que falte terminar algo que debiera estar: es una decisión deliberada, porque implementar el AST implica decisiones de diseño que todavía tenemos que cerrar como equipo, y preferimos no tomarlas de forma apurada o unilateral. La próxima etapa consiste en completar los cuerpos de **ast.c** y descomentar las acciones semánticas de **bison.y** una vez que terminemos de acordar esos últimos detalles.